TRACK IT

System Use Cases and Actors

Team Members and Assigned Use Cases

Name: Ahmed Ebrahem Hamdy **(PM)**

- **ID**: 23102354
- **Email**: Ahmed.abdelzaher.2024@aiu.edu.eg
- **Role:**

Subsystem

Use case Description: Manage Authentication

Test Requirements &Test Cases: User Authentication Management

2. **Name**: Mahmoud Emad Awad
   - **ID**: 23101474
   - **Email**: mahmoud.emad.2024@aiu.edu.eg
   - **Role:**

Use case diagram

Use case Description: Manage Payment

Test Requirements &Test Cases: Career Development System

3. **Name**: Muhammed Hany Hamdy
   - **ID**: 23102359
   - **Email**: Mohamed.abuaaliyah.2024@aiu.edu.eg
   - **Role:**

GUI

Use case Description: Manage Notifications

Test Requirements &Test Cases: Course Management

4. **Name**: Mostafa Khamis Abozead
    - **ID**: 23101484
    - **Email**: Mostafa Khamis Abozead
    - **Role**:

    Use case diagram

    Test Requirements &Test Cases: Course Management

5. **Name**: Mohamed Soliman Habib
    - **ID**: 23101462
    - **Email**: mohamed.abdelhamid.2024@aiu.edu.eg
    - **Role:**

    Use case Description: Manage Account

    Traceability Matrix

    Test Requirements &Test Cases: Manage RoadMap

1. Project Description

● The E-Learning Platform is an innovative, scalable, and secure digital ecosystem designed to revolutionize online education by delivering personalized learning experiences, advanced AI-driven tools, and seamless integration with career development opportunities. The platform empowers learners, instructors, and institutions to achieve their educational and professional goals through a unified, user-centric approach.

● Learners interact with adaptive AI tutors that analyze their performance, preferences, and career aspirations to generate tailored learning paths. The system tracks progress in real time, provides interactive quizzes with AI-powered grading, and offers hands-on coding labs and virtual projects to reinforce skill mastery. Gamification elements like badges, leaderboards, and streak tracking foster engagement and accountability.

● Instructors utilize advanced course creation tools, including AI-assisted content generation (e.g., auto-generated quizzes from lecture materials) and multi-format delivery (videos, SCORM packages, live webinars). Version control allows updates without disrupting enrolled students, while DRM and watermarking protect intellectual property. Instructors can monetize content through subscriptions, one-time purchases, or employer-sponsored courses, with automated financial reporting and payout management.

● Enterprise-Grade Security ensures compliance with global standards (GDPR, CCPA) through multi-factor authentication (MFA), OAuth logins, and role-based access control.

Suspicious activity detection, session expiration policies, and blockchain-verified certifications safeguard user data and content integrity.

● **Career Integration** bridges learning with job opportunities through AI-driven resume builders, skill gap analysis, and employer partnerships. The platform matches learners with internships and roles based on completed courses, while companies can sponsor courses aligned with in-demand skills.

● **A premium subscription**, accessible via integrated payment gateways (PayPal, Stripe), unlocks advanced features like live mentorship sessions, proctored exams, and priority access to hackathons. Institutions and employers receive dedicated dashboards to track learner progress, compliance, and ROI.

● **Financial Transparency**: The platform deducts a **15% service fee** from all transactions (course purchases, subscriptions). Instructors and mentors receive automated payouts for their earnings, with real-time tracking of revenue and deductions. Users view itemized billing for subscriptions, certifications, and mentorship sessions, ensuring clarity in all transactions.

Functional Requirements

User Authentication & Security

- Users can register and log in via email, password, and social media.
- Multi-Factor Authentication (MFA) and OAuth-based login supported.
- Secure session management with automatic expiration.
- Compliance with data privacy regulations (GDPR, CCPA).
- Suspicious activity detection and session revocation.

User Roles & Permissions

- Role-Based Access Control (RBAC) with custom permissions.
- Users can manage profiles, discussion posts, and activity feeds.
- Audit logs track user actions and data access.

Payments & Monetization

- Multiple payment gateway integration (PayPal, Stripe, etc.).

- Subscription plans, one-time purchases, and tiered access supported.
- Admins can create discounts, promo codes, and referral rewards.
- Financial reports and instructor payouts managed within the system.
- Wallet-based transactions and in-app purchases enabled.

Course Management & Delivery

- Instructors can create, edit, and organize courses into structured modules.
- Student progress tracking and adaptive difficulty settings.
- AI-assisted course creation (e.g., auto-generating quizzes).
- Multi-format content delivery (videos, PDFs, SCORM, interactive media).
- Certification issuance with public verification links.
- Course versioning support for content updates.

Learning Experience & Interaction

- Self-paced and instructor-led learning options.
- AI-powered grading for subjective assessments.
- Hands-on coding exercises, virtual labs, and project-based learning.
- Social learning features (discussion forums, messaging).

AI-Powered Learning & Personalization

- Adaptive learning paths based on performance and preferences.
- AI-powered course recommendations and skill gap analysis.
- Predictive analytics to improve retention.
- AI tutors and chatbots for 24/7 support.

Gamification & Engagement

- Points, badges, leaderboards, and level-based progressions.

- Seasonal challenges, streak tracking, and time-limited events.
- Collaborative projects and peer mentoring support.

Live Sessions & Webinars

- Real-time video conferencing for lectures and webinars.
- Interactive Q&A, polls, and chat moderation.
- AI-powered live transcriptions and translations.
- Playback access with chapter markers.

Hands-On Projects & Assessments

- Real-world case studies and capstone projects.
- Peer reviews and instructor feedback.
- Digital portfolio integration.
- Proctored exams and AI-assisted plagiarism detection.

Course Discovery & Recommendations

- AI-powered trending and personalized course suggestions.
- Structured learning paths with career-oriented roadmaps.
- Advanced search and filtering with NLP.

Career Support & Job Matching

- Users can create and manage career profiles.
- AI-powered resume builder with formatting suggestions.
- Direct job applications and resume uploads.
- Career counseling and skill assessments.
- AI-based job and internship recommendations.

Mentorship & Industry Networking

- AI-based mentor-mentee matching.

- One-on-one and group mentorship via video calls.
- Networking events and career fairs.

Hackathons & Competitions

- Hosting of coding challenges and hackathons.
- Solo and team participation support.
- Job opportunities and employer-sponsored contests.

Roadmap & Skill Progression

- Customizable learning tracks based on career goals.
- Prerequisite tracking and milestone achievements.

Admin Dashboard & Platform Management

- User, course, and financial management via dashboard.
- System health monitoring and real-time analytics.
- Audit logs and compliance tracking.

Mobile Learning & Offline Access

- Cross-platform support (iOS, Android, Web, PWA).
- Offline downloads with progress syncing.
- Push notifications for study reminders.

LMS & Third-Party Integrations

- API integration with external LMS platforms.
- Single Sign-On (SSO) and OAuth authentication.
- Webhooks and integrations (Slack, Zapier, Trello, LinkedIn, etc.).

Content Security & Intellectual Property Protection

- Digital Rights Management (DRM) and watermarking.
- Content integrity verification and access restrictions.

- Prevention of unauthorized sharing and screen recording.

AI & Automation Features

- Smart notifications and study reminders.
- Automated feedback on assignments.
- AI-powered resume optimization and job application assistance.

Certification & Accreditation System

- Stackable credentials and micro-degrees.
- Blockchain-based certificate verification.
- Employer recognition and job integration.

Non-Functional Requirements

Performance & Scalability

- The system shall handle at least **100,000 concurrent users** without performance degradation.
- Response time for key actions (login, course load, payment processing) shall not exceed **2 seconds**.
- The system shall scale dynamically based on user demand.

Security & Compliance

- The system shall comply with **GDPR, CCPA, and ISO 27001** security standards.
- User data shall be encrypted at rest and in transit using **AES-256 encryption**.
- System logs and security audits shall be maintained for **at least 12 months**.

Availability & Reliability

- The system shall maintain **99.9% uptime**, ensuring minimal downtime.

- Automatic failover mechanisms shall be implemented to prevent service interruptions.
- Scheduled maintenance shall be communicated to users at least **24 hours in advance**.

Usability & Accessibility

- The platform shall be **WCAG 2.1 AA compliant** for accessibility.
- All pages and components shall be mobile-responsive.
- The system shall support multiple languages for international users.

Major Use Cases

1.Manage Authentication

2. Manage User Roles

3. Manage Payments

4. Manage Courses

5. Manage Learning Experience

6. Manage AI-Powered Learning

7. Manage Achievements

8. Manage Live Sessions

9. Manage Career Support

10. Manage Mentorship

11. Manage Authorization

12. Manage Recommendations

13. Manage Certification

14. Manage Competitions

15. Manage RoadMap

16. Manage Platform

17. Manage Security

18.Manage Notifications

19.Manage Account

20.Manage Feedback

21.Manage Subscription

Actors List :

1. Student
2. Instructor
3. Admin
4. Employer
5. Mentor
6. Evaluation Repository System
7. Bank Repository System

Use Case Relationships

📌 Associations (↔)

These actors interact directly with the system.

Use Case	Associated Actors
Manage User Authentication	Student, Instructor, Admin
Manage User Authorization	Admin
Manage User Roles	Admin
Manage Payments	Student, Admin
Manage Subscription	Student

Manage Courses	Student, Instructor, Admin
Manage Learning Experience	Student, Instructor
Manage AI Progress	Student
Manage Achievements	Student
Manage Live Sessions	Instructor
Manage Career Support	Employer, Mentor
Manage Mentorship	Student, Employer, Mentor
Manage Recommendations	Student
Manage Certification	Student
Manage Competitions	Student
Manage RoadMap	Student
Manage Platform	Admin
Manage Security	Admin
Manage Notifications	Student, Instructor, Admin
Manage Account	Student, Instructor
Manage Feedback	Student

Include (+)

These use cases always require the included use cases.

Main Use Case	Includes (+)
Manage Payments	Manage Subscription
Manage User Authentication	Manage User Authorization
Manage Career Support	Manage Recommendations

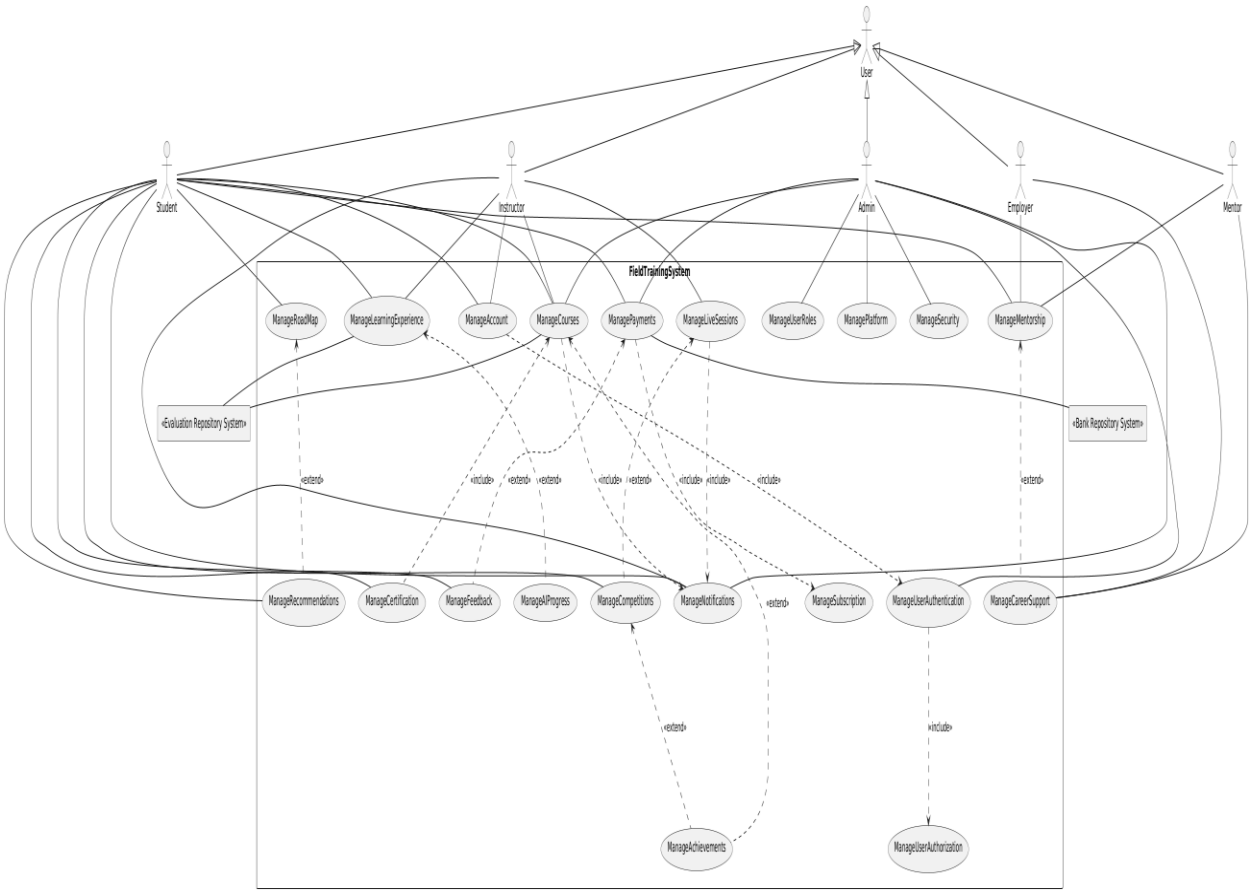
Manage Learning Experience	Manage AI Progress
Manage Live Sessions	Manage Notifications
Manage Courses	Manage Notifications, Manage Certification
Manage Account	Manage User Authentication
Manage RoadMap	Manage Recommendations

Extend (+)

These use cases optionally extend others based on specific conditions.

Base Use Case	Extends (+)
Manage Courses	Manage Feedback
Manage Learning Experience	Manage AI Progress
Manage Career Support	Manage Mentorship
Manage Live Sessions	Manage Competitions
Manage Competitions	Manage Achievements
Manage User Authentication	Manage User Authorization (if admin access is needed)

Structured Use Case Diagram:



Code:

@startuml

actor Student

actor Instructor

actor Admin

actor Employer

actor Mentor

Student -up-|> User

```
Instructor -up-|> User

Admin -up-|> User

Employer -up-|> User

Mentor -up-|> User


rectangle "<< Bank Repository System >>" as OrganizationRepoSystem

rectangle "<< Evaluation Repository System >>" as EvaluationRepoSystem


rectangle FieldTrainingSystem {


  usecase ManageUserAuthentication

  usecase ManageUserAuthorization

  usecase ManageUserRoles

  usecase ManagePayments

  usecase ManageSubscription

  usecase ManageCourses

  usecase ManageLearningExperience

  usecase ManageAIProgress

  usecase ManageAchievements

  usecase ManageLiveSessions

  usecase ManageCareerSupport

  usecase ManageMentorship

  usecase ManageRecommendations
```

```
usecase ManageCertification

usecase ManageCompetitions

usecase ManageRoadMap

usecase ManagePlatform

usecase ManageSecurity

usecase ManageNotifications

usecase ManageAccount

usecase ManageFeedback


' Use case relationships

ManagePayments -..-> ManageSubscription : <<include>>

ManageUserAuthentication -..-> ManageUserAuthorization : <<include>>

ManageLiveSessions -..-> ManageNotifications : <<include>>

ManageCourses <-..- ManageAchievements : <<extend>>

ManageLearningExperience <-..- ManageAIProgress : <<extend>>

ManageMentorship <-..- ManageCareerSupport : <<extend>>

ManagePayments <-..- ManageFeedback : <<extend>>

ManageCourses -..-> ManageNotifications : <<include>>

ManageLiveSessions <-..- ManageCompetitions : <<extend>>

ManageAccount -..-> ManageUserAuthentication : <<include>>

ManageCourses <-..- ManageCertification : <<include>>

ManageCompetitions <-..- ManageAchievements : <<extend>>

ManageRoadMap <-..- ManageRecommendations : <<extend>>
```

' Actors associations

Student -- ManagePayments

Student -- ManageCourses

Student -- ManageLearningExperience

Student -- ManageFeedback

Student -- ManageAccount

Student -- ManageCompetitions

Student -- ManageMentorship

Student -- ManageNotifications

Student -- ManageCertification

Student -- ManageRoadMap

Student -- ManageRecommendations

Instructor -- ManageCourses

Instructor -- ManageLearningExperience

Instructor -- ManageLiveSessions

Instructor -- ManageAccount

Instructor -- ManageNotifications

Admin -- ManageUserRoles

Admin -- ManagePayments

Admin -- ManageCourses

```
  Admin -- ManageUserAuthentication

  Admin -- ManagePlatform

  Admin -- ManageSecurity

  Admin -- ManageNotifications


  Employer -- ManageCareerSupport

  Employer -- ManageMentorship


  Mentor -- ManageMentorship

  Mentor -- ManageCareerSupport


  ' Repo systems

  ManagePayments -- OrganizationRepoSystem

  ManageCourses -- EvaluationRepoSystem

  ManageLearningExperience -- EvaluationRepoSystem


}


@enduml
```

Link: if photo not clear

Requirements:

- R1: Ensures secure user login with features like MFA, session management, and account lockout policies.
- R2: Handles scenarios such as incorrect credentials, password recovery, and security breaches.
- R3: Manages transactions, refunds, and billing while ensuring PCI-DSS compliance.
- R4: Implements fraud detection, encryption, and alternative payment methods.

- R5: Handles event-driven notifications via email, SMS, or in-app messages.
- R6: Supports user preferences, admin controls, and retry mechanisms for failed notifications.

Use Cases Specification:

Use Case Name

- **Manage Authentication**

Actors

- Customer (End User)
- Admin (System Administrator)

Type

- **Primary and essential**

Description

Authentication is a critical process that ensures secure access to the system by verifying the identity of users before granting them access. This use case describes the authentication mechanism for two types of users: **Customers** and **Admins**.

The authentication process begins when a user attempts to log in by providing valid credentials (e.g., username/email and password). The system validates these credentials against stored user data. Upon successful authentication, users are granted access to their respective dashboards based on their roles.

To enhance security, the authentication system includes additional mechanisms such as:

- **Multi-Factor Authentication (MFA):** An optional or mandatory second step requiring OTP verification via email, SMS, or an authenticator app.
- **Account Lockout Policies:** The system temporarily locks an account after a predefined number of failed login attempts to prevent brute-force attacks.
- **Session Management:** Once authenticated, the system generates a secure session token to maintain user access while preventing unauthorized session hijacking.

In case of a failed login attempt, the user is presented with an appropriate error message and can either retry or initiate the **password recovery** process if they have forgotten their credentials. If an account is locked due to excessive failed attempts, the user must follow a recovery process, which may involve verifying their identity via email or phone before regaining access.

Admins have additional security measures, such as **role-based access control (RBAC)**, ensuring that only authorized personnel can access sensitive administrative functions. The system may also enforce **IP whitelisting** or **geolocation-based access restrictions** for enhanced protection.

The authentication system is designed to be **secure, scalable, and user-friendly**, ensuring that users can seamlessly access the platform while preventing unauthorized access.

Cross References

- R1,R2

Preconditions

- The system is operational and accessible.
- The user has a registered account.
- Secure communication protocols (HTTPS, TLS) are enforced.

Postconditions

Success:

- The user is authenticated and redirected to the appropriate dashboard.
- A secure session is established.
- Login activity is logged for security purposes.

Failure:

- The user is denied access with an appropriate error message.
- Multiple failed attempts may trigger account lockout.
- Unauthorized access attempts may trigger security alerts.

Main Flow (Primary Scenario - Successful Login)

Actor Action	System Response
1-User navigates to the login page.	2-System displays the authentication form.

3-User enters username/email and password.	4-System validates credentials.
5-Credentials are correct.	6-System grants access and redirects to the appropriate dashboard.
7-System logs authentication details.	8-Secure session is established.

Alternative Scenarios

Incorrect Credentials

Actor Action	System Response
1-User enters invalid credentials.	2-System displays an error message.
3.User retries authentication.	4.If attempts exceed the limit, account is temporarily locked.

Account Locked Due to Multiple Failed Attempts

Actor Action	System Response
1.User exceeds failed login attempts.	2.System locks the account temporarily.
3.User tries to log in again.	4.System prompts for account recovery.
5.User follows recovery steps.	6.System unlocks account upon verification.

Forgot Password Flow

Actor Action	System Response
1.User selects "Forgot Password."	2.System prompts for email/phone number.
3.User enters details.	4.System sends a password reset link or OTP.
5.User resets the password.	6.System validates and allows login with new credentials.

Exception Handling

- **System Failure:** If authentication service is down, an error message is displayed, and users are notified.
- **Session Timeout:** If a session expires due to inactivity, the system prompts reauthentication.
- **Security Breach Detection:** If multiple failed attempts are detected from different IP addresses, the system triggers an alert.

Interaction Scenarios

- A **customer** logs in : Redirected to the **user dashboard**.
- An **admin** logs in : Redirected to the **admin panel**.
- If authentication **fails**, an error message is displayed with retry options.
- If **MFA is enabled**, an additional verification step is required.

Alternative Courses

- **User attempts to log in from a restricted IP : Access denied.**
- **User enters old credentials after a forced password reset : Prompted to update password.**
- **User session is hijacked : System detects anomaly and logs out user.**

Use Case Name

Manage Payment

Actors

- Customer (End User)
- Admin (System Administrator)
- Payment Gateway(<<External Bank Rep System>>)

Type

- **Primary and essential**

Description

The Manage Payment use case describes the process of handling payments made by customers for services or products. It ensures that transactions are processed securely, and payments are recorded accurately. Customers can make payments using various methods such as credit/debit cards, digital wallets, or bank transfers. Admins have access to payment records, refunds, and transaction management features.

The payment process includes the following key components:

- **Payment Authorization:** The system validates payment details and checks for available funds before proceeding.
- **Transaction Processing:** The payment gateway securely processes the transaction and confirms success or failure.
- **Invoice Generation:** After a successful payment, an invoice is generated and sent to the customer.
- **Refund and Dispute Management:** Admins can process refunds and handle payment disputes if necessary.

- **Security Measures:** The system ensures encryption of payment data and compliance with PCI-DSS standards.

If a payment fails, the system notifies the customer and provides alternative payment options. Admins can review payment logs and transaction statuses to resolve issues efficiently.

Cross References

- R3,R4

Preconditions

- The system is operational and accessible.
- The user has a valid account.
- The payment gateway is available and functional.

Postconditions

Success:

- Payment is successfully processed.
- The customer receives a confirmation and invoice.
- The system updates the payment records.

Failure:

- The transaction fails, and an error message is displayed.
- The system provides alternative payment options.
- The system logs the failed transaction for review.

Main Flow (Primary Scenario - Successful Payment)

Actor Action	System Response
1.Customer selects items/services and proceeds to checkout.	2.System displays payment options.
3.Customer enters payment details.	4.System validates details and checks for available funds.
5.System sends payment request to the payment gateway.	6.Payment gateway processes the transaction.
7.Payment is approved.	8.System confirms the transaction and updates records.
9.System generates an invoice.	10.Customer receives confirmation and invoice.

Alternative Scenarios

Payment Declined

Actor Action	System Response
1.Customer enters payment details.	2.System validates details and detects insufficient funds.
3.System declines the transaction.	4.Error message is displayed with retry options.
5.Customer chooses an alternative payment method.	6.System reprocesses the transaction.

Refund Process

Actor Action	System Response
1.Admin initiates a refund request.	2.System verifies the transaction.
3.Refund request is approved.	4.System processes the refund via the payment gateway.
5.Customer receives a refund confirmation.	6.System updates payment records.

Exception Handling

- **Payment Gateway Failure:** If the payment gateway is down, the system notifies the customer and suggests retrying later.
- **Session Timeout:** If the payment session expires, the system prompts the user to restart the process.
- **Fraud Detection:** If suspicious activity is detected, the transaction is flagged, and additional verification is required.

Interaction Scenarios

- A **customer** completes a payment : Receives confirmation and invoice.
- An **admin** reviews transaction logs : Processes refunds or disputes if necessary.
- If **payment fails**, the system provides retry and alternative payment options.
- If **fraud detection triggers**, additional security verification is required.

Alternative Courses

- **Customer uses a promo code** : System applies the discount before payment.
- **Customer cancels the payment before submission** : Transaction is aborted.
- **Admin adjusts a payment record** : System logs the change for audit purposes.

Use Case Name

- **Manage Notification**

Actors

- Customer (End User)
- Admin (System Administrator)

Type

- **Primary and essential**

Description

The Manage Notification use case describes the process of handling system-generated notifications for customers and admins. The notification system ensures timely communication of important updates, alerts, and transactional information. Notifications can be triggered by various system events and delivered through multiple channels, such as email, SMS, or in-app messages.

The notification process includes the following key components:

- **Event-Triggered Notifications:** Notifications are generated based on system activities such as payment confirmation, authentication alerts, or account updates.
- **User Preferences:** Customers can configure notification settings, choosing preferred channels and types of alerts.
- **Admin Controls:** Admins can send broadcast notifications, manage notification templates, and monitor delivery status.
- **Security Measures:** Notifications involving sensitive data are encrypted, and access to notification logs is restricted based on user roles.

If a notification fails to send, the system retries delivery and logs the failure for review. Admins can track notification history and troubleshoot issues as needed.

Cross References

- R5, R6

Preconditions

- The system is operational and accessible.
- The user has a valid account.
- The notification service is enabled and functional.

Postconditions

Success:

- Notification is successfully delivered to the user.
- The system logs the notification event.

Failure:

- The notification fails to send, and a retry attempt is scheduled.
- The system logs the failure for review.

Main Flow (Primary Scenario - Successful Notification Delivery)

Actor Action	System Response
1.System detects an event requiring notification.	2.System generates the notification message.
3.System selects the appropriate delivery channel.	4.Notification is sent via email, SMS, or in-app message.
5.Notification is successfully delivered.	6.System logs the notification event.

Alternative Scenarios

Notification Failure

Actor Action	System Response
1.System generates a notification.	2.Attempt to send notification fails.
3.System retries notification delivery.	4.Notification is resent via alternative channels if configured.
5.Notification still fails.	6.System logs failure and alerts admin.

User Opts Out of Notifications

Actor Action	System Response
1.Customer updates notification preferences.	2.System updates user settings.
3.System generates an event that triggers a notification.	4.System checks user preferences and suppresses notification if opted out.

Exception Handling

- **Notification Service Down:** If the notification service is unavailable, the system queues notifications for later delivery.
- **Invalid Contact Information:** If an email or phone number is incorrect, the system logs the issue and prompts the user to update contact details.
- **High Volume Notification Load:** If too many notifications are triggered simultaneously, the system prioritizes urgent messages and processes others in a queue.

Interaction Scenarios

- A **customer** receives a payment confirmation notification.

- An **admin** sends a system-wide announcement.
- A **customer** disables promotional notifications but continues to receive security alerts.
- A **notification fails**, and the system retries delivery before logging an issue.

Alternative Courses

- **User blocks notifications from an email provider** → System detects undelivered emails and suggests updating settings.
- **Admin configures priority notifications** → System ensures urgent messages are sent first.
- **User reads an in-app notification** → System marks it as "Read" and updates the notification center.

Use Case: Manage Account

- 1. General Information
- Actor:
  - Customer
- Trigger:
  The customer initiates the process by selecting the “Manage Account” option, typically when they wish to update their personal or payment information.
- Preconditions:
  - The customer is authenticated (logged in).
  - The customer’s session is active.
  - The system has access to the most recent account data.
- Postconditions:
  - The updated account information is saved to the customer profile.
  - Changes are immediately available for subsequent transactions and interactions.
-

2. Main Flow

- Initiate Process:
  - The customer selects the “Manage Account” option from the profile menu.
- Retrieve Data:

    - The system retrieves current account details from the database, including personal information, saved addresses, and payment methods.
- **Display Information:**
    - The system displays the account details on a user-friendly interface, structured into sections (e.g., Personal Info, Addresses, Payment Methods).
- **Edit Account Details:**
    - The customer navigates to the relevant sections and edits necessary fields (e.g., updating an address or payment method).
- **Submit Updates:**
    - Once the edits are complete, the customer submits the changes via a "Save" or "Update" button.
- **Validation:**
    - The system validates the submitted data:
        - Checks for required fields.
        - Ensures data formats (e.g., email, phone number) comply with predefined rules.
        - Verifies that there are no conflicts (e.g., duplicate addresses).
- **Data Persistence:**
    - If validation passes, the system updates the customer profile in the database.
- **Confirmation:**
    - The system confirms the successful update with a confirmation message and immediately displays the updated account details.
-

3. Alternative Flows

- **A. Invalid Data Input**
- **Scenario:** The customer enters data that fails validation.
- **Steps:**
    - The system detects invalid or incomplete data during validation.
    - An error message is displayed next to the problematic fields, indicating the specific issues (e.g., "Invalid email format").
    - The customer corrects the errors and resubmits the updated information.
    - The system re-validates and proceeds with the update if all issues are resolved.
- **B. Session Timeout**

- **Scenario:** The customer's session expires during the update process.
- **Steps:**
    - The system detects the session timeout either during data entry or upon submission.
    - A prompt is displayed asking the customer to log in again.
    - After successful re-authentication, the system may restore unsaved changes or request re-entry of data.
- **C. Network/Database Connectivity Issues**
- **Scenario:** The system is unable to connect to the database when saving changes.
- **Steps:**
    - The system displays an error message indicating connectivity issues.
    - The customer is advised to try submitting again after a short delay.
    - An error log is created for system administrators to review and address connectivity issues.
-
- **4. Exception Flow**
- **Unexpected Errors:**
    - **If an unexpected error occurs during data retrieval or saving:**
        - The system logs the error details (with a reference number) for further investigation.
        - A generic error message is displayed to the customer (e.g., "An error occurred while updating your account. Please try again later or contact support with reference number XYZ123").
        - The customer is given an option to contact support for assistance.
-
- **5. Business Rules & Non-Functional Requirements**
- **Business Rules:**
    - Only authenticated users can access and update account information.
    - All updates must pass validation checks to ensure data integrity.
    - Sensitive data (e.g., payment methods) should be masked or encrypted in the user interface.
    - Every update should trigger an audit log entry for tracking changes.
- **Non-Functional Requirements:**
    - The system should reflect account updates in real-time (ideally within 2 seconds).

- The interface should be responsive and accessible across various devices (desktop, mobile, tablet).
- Data security protocols must be followed to protect customer information.

-
- **6. Assumptions**
- The customer has previously verified their account details (e.g., verified email address).
- The system's backend and database are operational and updated in real-time.
- Error handling procedures are in place to manage both expected and unexpected issues during the update process.

Subsystem Table:

Subsystem Name	Subsystem Function	Subsystem Interface

1. Authentication Management	**User Registration**
	• **Allow new users to register by providing personal details, username, password, and optional 2FA setup.**
	User Login
	• **Authenticate users using their credentials (e.g., username/email + password).**
	• **Support multi-factor authentication (MFA) if enabled.**
	Password Management
	• **Change password functionality.**
	• **Forgot password / reset password via email or phone verification.**
	Session Management
	• **Manage user sessions with secure tokens (JWT, session ID, etc.).**
	• **Allow logout from all sessions.**
	• **Auto-expire inactive sessions.**
	Role-Based Access Control (RBAC)
	• **Assign roles (e.g., Admin, User, Moderator) to users.**
	• **Restrict access to resources based on user roles and permissions.**
	Account Lockout & Brute Force Protection
	• **Lock accounts after multiple failed login attempts.**
	• **Use CAPTCHA after suspicious login behavior.**
	Authentication Logs & Monitoring
	• **Log all login attempts (successful & failed).**
	• **Track IP address, timestamp, and device info.**
	OAuth/Social Login Integration
	• **Allow users to log in using Google, Facebook, GitHub, etc.**
	Token Management
	• **Issue and refresh tokens for authenticated sessions.**
	• **Revoke tokens upon logout or suspicious activity.**
	Email & Phone Verification
	• **Send verification codes/links to confirm user identity**
		// User Registration and Authentication
		public interface AuthService {
		AuthResponse login(String username, String password);
		void logout(String token);
		void register(UserRegistrationRequest request);
		void resetPassword(ResetPasswordRequest request);
		void changePassword(ChangePasswordRequest request);
		boolean verifyTwoFactorCode(String userId, String code);
		}
		// Session Management
		public interface SessionService {
		List<SessionInfo> getActiveSessions(String userId);
		void terminateSession(String sessionId);
		void terminateAllSessions(String userId);
		}
		// Role and Permission Management
		public interface RoleService {
		List<Role> getAllRoles();
		void assignRoleToUser(String userId, String roleId);
		List<Permission> getPermissionsByRole(String roleId);
		}
		// Notification (Email/SMS for verification or reset)
		public interface NotificationService {
		void sendEmail(String to, String subject, String content);
		void sendSms(String phoneNumber, String message);
		}
		// OAuth/Social Login Integration
		public interface OAuthService {
		AuthResponse authenticateWithProvider(String provider, String accessToken);
		}

2. Authorization Management	**Role Management**
	• Create Role
	• Update Role
	• Delete Role
	• List All Roles
	• Get User Roles
	• Assign Role to User
	• Remove Role from User
	Permission Management
	• Create Permission
	• Update Permission
	• Delete Permission
	• Assign Permission to Role
	• Remove Permission from Role
	• List Role Permissions
	• Get User Effective Permissions
	Access Control Evaluation
	• Check User Authorization
	• Evaluate Access Policy
	• Get Accessible Resources for User
	Policy Management (Optional)
	• Create Access Policy
	• Attach Policy to Role
	• Evaluate Custom Policy
	Audit Logging & Monitoring
	• Log Authorization Attempt
	• Retrieve Authorization Logs
	• View Role Change History
	• View Permission Audit Trail
		// Role operations
		public interface RoleService {
		void createRole(String roleName);
		void updateRole(String roleId, String newName);
		void deleteRole(String roleId);
		List<Role> getAllRoles();
		List<Role> getUserRoles(String userId);
		void assignRoleToUser(String userId, String roleId);
		void removeRoleFromUser(String userId, String roleId);
		}
		// Permission operations
		public interface PermissionService {
		void createPermission(String permissionName);
		void assignPermissionToRole(String roleId, String permissionId);
		void removePermissionFromRole(String roleId, String permissionId);
		List<Permission> getPermissionsByRole(String roleId);
		boolean userHasPermission(String userId, String permissionName);
		}
		// Access control checker (authorization engine)
		public interface AccessControlService {
		boolean isAccessAllowed(String userId, String resource, String action);
		List<String> getAccessibleResources(String userId);
		}

3. Payment Processing	**Initiate Payment**
	• Starts the payment process for a transaction.
	• Validates input like amount, currency, and payment method.
	
	Process Payment
	• Handles communication with external payment gateways (e.g., Stripe, PayPal, Visa, etc.).
	• Applies business logic like discounts or taxes before submission.
	
	Verify Payment
	• Confirms whether the payment was successful or failed.
	• Handles payment confirmation callbacks (e.g., from payment gateways).
	
	Record Transaction
	• Logs the transaction in the system's database.
	• Links payments to user accounts/orders/invoices.
	
	Handle Refunds
	• Supports full or partial refund operations.
	• Updates records and notifies users.
	
	Generate Receipts
	• Creates digital receipts (PDF/email) for successful transactions.
	• Sends to the user via email or downloads.
	
	Payment History Retrieval
	• Allows users or admins to retrieve past payment records.
	• Supports filtering by date, amount, status, etc.
	
	Fraud Detection
	• Flags suspicious activities or payment anomalies.
	• Optionally integrates with a risk analysis engine.
		public interface PaymentProcessing {
		
		// 1. Initiate a new payment
		
		String initiatePayment(String userId, double amount, String method);
		
		// 2. Process payment with a payment ID (e.g., calling external gateway)
		
		boolean processPayment(String paymentId);
		
		// 3. Verify payment status after processing
		
		boolean verifyPayment(String paymentId, String gatewayResponse);
		
		// 4. Record the transaction details in the database
		
		void recordTransaction(String userId, PaymentDetails details);
		
		// 5. Issue a refund for a given payment
		
		boolean refundPayment(String paymentId, double amount);
		
		// 6. Retrieve user's payment history with optional filters
		
		List<PaymentDetails> getPaymentHistory(String userId, PaymentFilter filter);
		
		// 7. Generate a receipt (could return a file path or byte array)
		
		byte[] generateReceipt(String paymentId);
		
		// 8. Run fraud detection logic
		
		boolean checkFraud(String paymentId);
		
		}

4. Course Management	**Create Course**
	
	6. Allows administrators or instructors to create new courses.
	7. Includes details like course title, description, syllabus, and prerequisites.
	
	Edit Course
	
	8. Allows updating course details, including adding/removing modules, changing content, or updating prerequisites.
	
	Delete Course
	
	9. Allows removal of a course from the system, including associated materials.
	
	Enroll in Course
	
	10. Allows students to enroll in available courses.
	11. Checks for prerequisites, availability, and the student's eligibility.
	
	Assign Instructors
	
	12. Assigns instructors to specific courses.
	13. Includes managing multiple instructors for a single course if needed.
	
	View Course Details
	
	14. Retrieves all information about a course (syllabus, prerequisites, instructor details, etc.).
	
	Manage Course Materials
	
	15. Upload, modify, or remove course materials like videos, PDFs, or assignments.
	
	Track Course Progress
	
	16. Monitors the progress of enrolled students.
	17. Updates completion status, scores, and feedback.
	
	Grade Assignments/Exams
	
	18. Allows instructors to grade assignments or exams within the course.
	
	Generate Course Reports
	
	19. Provides statistics like student progress, grades, and course performance.
		public interface CourseManagement {
		
		// Create a new course
		
		boolean createCourse(Course course);
		
		// Edit an existing course
		
		boolean editCourse(String courseId, Course updatedCourse);
		
		// Delete a course
		
		boolean deleteCourse(String courseId);
		
		// View course details
		
		Course viewCourseDetails(String courseId);
		
		}
		
		public interface Enrollment {
		
		// Enroll a student in a course
		
		boolean enrollStudent(String studentId, String courseId);
		
		// View enrolled students in a specific course
		
		List<String> viewEnrolledStudents(String courseId);
		
		}
		
		public interface CourseContent {
		
		// Add or remove course material
		
		boolean manageCourseMaterial(String courseId, Material material, boolean add);
		
		// View all course materials for a specific course
		
		List<Material> viewCourseMaterials(String courseId);
		
		}
		
		public interface Grading {
		
		// Grade a student's assignment or exam
		
		boolean gradeAssignment(String studentId, String courseId, String assignmentId, double grade);
		
		// View student grades in a course
		
		List<Grade> viewStudentGrades(String studentId, String courseId);
		
		}
		
		public interface Reporting {

		// Generate course performance report
		
		List<CourseReport> generateCourseReport(String courseId);
		
		// Generate individual student progress report in a course
		
		StudentProgressReport generateStudentReport(String studentId, String courseId);
		
		}

5. Learning Experience Management	1. **Track Learner Progress**
	○ Monitors the learner's progress through courses, lessons, or modules.
	○ Updates progress based on completed tasks, quizzes, or assignments.
	2. **Provide Personalized Recommendations**
	○ Recommends courses, lessons, or resources based on the learner's interests, performance, or learning path.
	3. **Manage Learning Paths**
	○ Defines and manages structured learning paths for learners, guiding them through a series of courses/modules.
	○ Allows customization of learning paths based on specific goals.
	4. **Gamification and Rewards**
	○ Implements gamified elements such as points, badges, or leaderboards to motivate learners.
	○ Allows learners to unlock rewards as they progress through content.
	5. **Engage Learners with Notifications**
	○ Sends notifications to learners for new content, assignments, or upcoming deadlines.
	○ Reminds learners of pending activities or tasks.
	6. **Provide Feedback and Assessments**
	○ Allows instructors to give feedback on assignments or quizzes.
	○ Provides automated feedback on performance to learners.
	7. **Manage Learning Groups or Communities**
	○ Facilitates collaborative learning through discussion groups, forums, or peer reviews.
	○ Manages learner groups for team-based assignments or projects.
	8. **Track Learner Engagement**
	○ Monitors how actively learners interact with the platform (e.g., login frequency, lesson completion).
	○ Provides engagement reports to instructors or admins.
	9. **Surveys and Evaluations**
	○ Collects feedback from learners on courses, instructors, and the learning platform.
	○ Allows learners to rate or review courses and content.
		public interface LearningExperienceManagement {
		
		// 1. Track learner's progress in a course
		
		void trackLearnerProgress(String learnerId, String courseId, double progress);
		
		// 2. Provide personalized course recommendations for a learner
		
		List<String> recommendCourses(String learnerId);
		
		// 3. Create or manage learning paths for learners
		
		boolean createLearningPath(String learnerId, List<String> courseIds);
		
		// 4. Implement gamification elements (e.g., points, badges)
		
		boolean rewardLearner(String learnerId, String rewardType);
		
		// 5. Send notifications to learners about new content, tasks, or deadlines
		
		void sendNotification(String learnerId, String message);
		
		// 6. Provide feedback on learner's performance (assignments, exams, etc.)
		
		boolean provideFeedback(String learnerId, String courseId, String feedback);
		
		// 7. Manage learner groups for collaboration or assignments
		
		boolean createLearningGroup(String groupName, List<String> memberIds);
		
		// 8. Track learner engagement (e.g., activity, time spent)
		
		EngagementStats trackEngagement(String learnerId);
		
		// 9. Send surveys or evaluations to learners
		
		boolean sendSurvey(String learnerId, String surveyId);}

6. Live Learning Platform	1. **Host Live Sessions** • Allows instructors to create and host live sessions or webinars. • Supports features like video/audio streaming, screen sharing, and live chats. 2. **Join Live Sessions** • Allows learners to join live sessions or webinars. • Provides real-time interaction via chat, reactions, or polls. 3. **Live Chat & Q&A** • Enables learners to ask questions and interact with instructors during live sessions. • Supports private messages between learners and instructors. 4. **Breakout Rooms**	public interface LiveLearningPlatform // 1. Host a new live session boolean hostLiveSession(String instructorId, SessionDetails sessionDetails); // 2. Join an existing live session

- Allows instructors to create breakout rooms for smaller group discussions during live sessions.
- Supports real-time collaboration and group activities.

5. **Session Recording**
   - Records live sessions for later access by learners.
   - Provides a video archive of recorded sessions for review.
6. **Interactive Polls and Quizzes**
   - Allows instructors to create interactive polls or quizzes during live sessions.
   - Provides immediate feedback and results to the learners.
7. **Attendance Tracking**
   - Tracks the attendance of learners in live sessions.
   - Provides attendance reports for instructors.
8. **Live Session Notifications**
   - Sends notifications to learners about upcoming live sessions, reminders, or changes in schedule.
9. **Whiteboard and Collaboration Tools**
   - Provides virtual whiteboards for instructors to write or draw in real-time.
   - Includes other collaborative tools like document sharing and annotations.
10. **Record and Share Resources**
    - Allows instructors to upload and share learning resources during or after the session.

```
    boolean joinLiveSession(String learnerId, String sessionId);

    // 3. Enable live chat and Q&A during the session

    boolean sendMessage(String sessionId, String learnerId, String message);

    // 4. Create breakout rooms for group discussions

    boolean createBreakoutRoom(String sessionId, List<String> participantIds);

// 5. Record a live session for future access

    boolean recordSession(String sessionId);

    // 6. Create interactive polls or quizzes during the session

    boolean createPoll(String sessionId, String pollQuestion, List<String> options);

    // 7. Track attendance of learners in a live session

    boolean trackAttendance(String sessionId, String learnerId);

    // 8. Send notifications to learners about live sessions

    void sendNotification(String learnerId, String message);

    // 9. Share resources (documents, slides, etc.) during the session

    boolean shareResources(String sessionId, Resource resource);

    // 10. Provide a whiteboard for collaborative learning

    boolean useWhiteboard(String sessionId, String content);

}
```

7. Project Assessment System	**Create Project Assignments**
	Allows instructors to create project assignments, specifying the requirements, deadlines, and evaluation criteria.
	Submit Project
	Allows learners to submit their projects or assignments within a given timeframe.
	Supports file uploads, code submissions, and other project artifacts.
	Peer Review
	Enables learners to review and assess each other's projects based on predefined criteria.
	Supports anonymous reviews to ensure fairness.
	Grade Projects
	Allows instructors or automated systems to grade projects based on predefined rubrics.
	Provides detailed feedback on the project's strengths and areas for improvement.
	Provide Feedback
	Instructors provide written or video feedback for submitted projects.
	Learners can access feedback for improving future projects.
	Track Project Progress
	Allows both instructors and learners to track the progress of project completion.
	Provides status updates on submitted, in-progress, and incomplete projects.
	Automated Scoring
	Supports automated grading for projects that involve code or standard input-output evaluation.
	Compares project output against expected results to provide a score.
	Project Evaluation Rubrics
	Allows instructors to define custom evaluation rubrics based on various factors like design, implementation, documentation, and functionality.
	Project Revision and Resubmission
	Enables learners to revise their projects based on feedback and resubmit them for further evaluation.
	Generate Project Reports
	Provides detailed reports on learner project submissions, grades, and feedback.
	Generates performance reports for instructors to evaluate class performance
		public interface ProjectAssessmentSystem {
		// 1. Create a new project assignment
		boolean createProjectAssignment(String instructorId, ProjectAssignment assignment);
		// 2. Submit a project for evaluation
		boolean submitProject(String learnerId, String projectId, ProjectSubmission submission);
		// 3. Peer review for a project submission
		boolean peerReviewProject(String reviewerId, String projectId, Review review);
		// 4. Grade a project submission
		boolean gradeProject(String instructorId, String projectId, double grade, String feedback);
		// 5. Provide feedback on project submission
		boolean provideFeedback(String instructorId, String projectId, String feedback);
		// 6. Track progress of project submission
		ProjectProgress trackProjectProgress(String learnerId, String projectId);
		// 7. Automate project grading (for code-based projects)
		boolean automateGrading(String projectId, String testData)
		// 8. Define and manage evaluation rubrics for grading
		boolean createEvaluationRubric(String projectId, EvaluationRubric rubric);
		// 9. Enable project revision and resubmission
		boolean allowRevision(String learnerId, String projectId, ProjectSubmission newSubmission);
		// 10. Generate project reports for instructor and learner
		ProjectReport generateProjectReport(String learnerId, String projectId);
		}

8. Course Recommendation System	1. **User Profile Analysis**
	• Analyzes the learner's profile, including previous courses completed, interests, skills, and career goals, to understand their preferences.
	2. **Recommend Courses Based on User Data**
	• Recommends courses based on the learner's interests, skills, and learning history.
	• Uses collaborative filtering, content-based filtering, or hybrid approaches to make recommendations.
	3. **Personalized Course Path**
	• Suggests a sequence of courses to help learners achieve specific learning objectives or career goals.
	4. **Trending Courses**
	• Recommends popular or trending courses among users, either globally or within a specific group or community.
	5. **Skill Gap Analysis**
	• Identifies skills the learner may need to improve based on their past learning and current goals, and suggests courses to fill those gaps.
	6. **Course Reviews and Ratings**
	• Recommends courses based on high ratings and positive feedback from other learners.
	7. **Courses Based on Learning Pace**
	• Recommends courses based on the learner's preferred learning speed, such as self-paced or instructor-led courses.
	8. **Recommend Certifications**
	• Suggests certification programs based on the learner's progress and career goals.
	9. **Course Difficulty Level Adjustment**
	• Recommends courses based on the learner's current expertise level (beginner, intermediate, advanced).
	10. **Dynamic Updates**
	• Updates course recommendations dynamically as the learner completes new courses or changes their preferences.
		public interface CourseRecommendationSystem {
		
		// 1. Analyze learner profile and preferences
		
		UserProfile analyzeUserProfile(String learnerId);
		
		// 2. Recommend courses based on learner's interests, skills, and learning history
		
		List<String> recommendCourses(String learnerId);
		
		// 3. Recommend a personalized learning path to achieve goals
		
		List<String> recommendLearningPath(String learnerId, String goal);
		
		// 4. Recommend trending courses
		
		List<String> recommendTrendingCourses();
		
		// 5. Suggest courses based on skill gap analysis
		
		List<String> recommendCoursesBasedOnSkillGap(String learnerId);
		
		// 6. Suggest courses with high ratings and positive reviews
		
		List<String> recommendTopRatedCourses();
		
		// 7. Recommend courses based on learning pace preference
		
		List<String> recommendCoursesByLearningPace(String learnerId);
		
		// 8. Recommend certification programs based on career goals
		
		List<String> recommendCertifications(String learnerId);
		
		// 9. Suggest courses based on learner's current expertise level
		
		List<String> recommendCoursesByDifficultyLevel(String learnerId);
		
		// 10. Update course recommendations based on new learner activity
		
		List<String> updateRecommendations(String learnerId);
		
		}

9. Certification Management	1. **Issue Certification**
	• Issues certifications to learners upon successful completion of a course or program.
	• Can include details like learner name, course title, completion date, and instructor information.
	2. **Verify Certification**
	• Allows users (such as employers or third-party organizations) to verify the authenticity of a learner's certification using a unique certificate ID or QR code.
	3. **Track Certification Status**
	• Tracks whether a learner has completed the necessary requirements for a certification, including courses and exams.
	4. **Store Certification Records**
	• Maintains a record of all issued certifications, including details such as issuance date, validity, and associated course.
	5. **Expiration and Renewal**
	• Manages certification expiration and renewal processes, including notifying learners when their certificates are close to expiration.
	6. **Certification Revocation**
	• Allows for the revocation of a certification if the learner violates terms or fails to maintain qualifications.
	7. **Customized Certificate Templates**
	• Provides customizable templates for certifications, allowing for branding and personalization based on different courses or programs.
	8. **Bulk Certificate Issuance**
	• Enables the bulk issuance of certificates for a group of learners upon completion of a course or program.
	9. **Generate Certification Reports**
	• Generates detailed reports for instructors and administrators, including a list of learners who have earned certifications and their respective details.
	10. **Track Certification Progress**
	• Allows administrators and learners to track progress towards certification requirements, such as course completions, assessments, and prerequisites.
		public interface CertificationManagementSystem {
		// 1. Issue a certificate to a learner upon course completion
		boolean issueCertificate(String learnerId, String courseId);
		// 2. Verify the authenticity of a certification
		boolean verifyCertification(String certificateId);
		// 3. Track the status of certification requirements
		CertificationStatus trackCertificationStatus(String learnerId, String certificationId);
		// 4. Store and maintain records of issued certifications
		boolean storeCertificationRecord(Certificate certificate);
		// 5. Manage certificate expiration and renewal
		boolean manageCertificateExpiration(String learnerId, String certificateId);
		// 6. Revoke certification if necessary
		boolean revokeCertification(String certificateId);
		// 7. Customize certificate templates based on course or program
		boolean customizeCertificateTemplate(String courseId, CertificateTemplate template);
		// 8. Bulk issue certificates to a group of learners
		boolean bulkIssueCertificates(List<String> learnerIds, String courseId);
		// 9. Generate a report of issued certifications
		List<CertificateReport> generateCertificationReport(String courseId);
		// 10. Track progress toward certification completion
		CertificationProgress trackCertificationProgress(String learnerId, String certificationId);
		}

10. Career Development System	1. **Career Path Analysis** • Analyzes current market trends and provides career advice based on the learner's educational and work history. 2. **Skills Assessment and Evaluation** • Assesses current skills and provides recommendations for improvement, including suggesting relevant courses to bridge skill gaps. 3. **Professional Reporting** • Provides detailed reports on professional progress, potential promotions, and areas for improvement. 4. **Job Matching** • Matches learners with job opportunities based on their skills, career goals, and qualifications. 5. **Resume and Portfolio Building** • Allows learners to create or update resumes and portfolios, highlighting their skills, achievements, and qualifications. 6. **Interview Preparation**	public interface CareerDevelopmentSystem { // 1. Recommend career paths based on learner's skills and interests List<String> recommendCareerPaths(String learnerId); // 2. Match job opportunities based on learner's skills and career goals List<Job> recommendJobOpportunities(String learnerId); // 3. Allow learners to build or update their resumes and portfolios

	• Offers resources and tools for interview preparation, including mock interviews, sample questions, and interview tips.
	7. **Networking Opportunities**
	• Connects learners with professionals and employers through networking events, webinars, and online communities.
	8. **Internship and Apprenticeship Opportunities**
	• Provides access to internships and apprenticeships to help learners gain real-world experience and build their resumes.
	9. **Job Search Tools**
	• Provides a job search engine with filters to help learners find job openings that match their skills and interests.
	10. **Career Coaching and Mentorship**
	• Offers career coaching services where learners can connect with professionals for guidance and mentorship.
	11. **Career Progress Tracking**
	• Tracks the learner's career development, including job applications, interviews, offers, and certifications earned.
		boolean createOrUpdateResume(String learnerId, Resume resume);
		
		// 4. Provide job interview preparation resources
		
		boolean prepareForInterview(String learnerId);
		
		// 5. Connect learners with networking opportunities
		
		List<String> findNetworkingOpportunities(String learnerId);
		
		// 6. Recommend internships or apprenticeships
		
		List<String> recommendInternships(String learnerId);
		
		// 7. Provide job search tools and filters
		
		List<Job> searchJobs(String learnerId, JobSearchCriteria criteria);
		
		// 8. Offer career coaching and mentorship services
		
		boolean offerCareerCoaching(String learnerId);
		
		// 9. Track career development progress and milestones
		
		CareerProgress trackCareerProgress(String learnerId);
		
		}

11. Mentorship Management	**Mentor-Mentee Matching**
	• Matches mentors with mentees based on their skills, interests, career goals, and expertise.
	• Uses algorithms to ensure optimal pairings between mentors and mentees.
	
	Mentorship Program Enrollment
	• Allows learners and professionals to enroll in the mentorship program either as a mentor or mentee.
	• Collects and stores data such as skills, experience, and preferences for personalized matchmaking.
	
	Communication Channels
	• Provides secure communication channels (e.g., messaging, video calls) between mentors and mentees.
	• Supports scheduled meetings, reminders, and meeting agendas.
	
	Mentorship Goal Setting
	• Allows mentors and mentees to set clear, measurable goals for their mentorship relationship.
	• Tracks the progress of the mentee's development toward these goals.
	
	Session Scheduling and Management
	• Allows mentors and mentees to schedule meetings, whether virtual or in person.
	• Provides a calendar integration for easy tracking and notifications.
		public interface MentorshipManagementSystem {
		
		// 1. Match mentors with mentees based on skills and interests
		
		MentorMenteePair matchMentorMentee(String menteeId);
		
		// 2. Enroll mentors and mentees in the mentorship program
		
		boolean enrollInProgram(String userId, String role); // role can be "mentor" or "mentee"
		
		// 3. Enable secure communication between mentors and mentees
		
		boolean initiateCommunication(String mentorId, String menteeId, String message);
		
		// 4. Allow mentors and mentees to set goals for their relationship
		
		boolean setMentorshipGoals(String mentorId, String menteeId, List<String> goals);
		
		// 5. Schedule mentorship sessions
		
		boolean scheduleSession(String mentorId, String menteeId, String dateTime);
		
		// 6. Track mentorship progress and provide feedback
		
		boolean trackProgress(String mentorId, String menteeId, String feedback);

Mentorship Progress Tracking

- Tracks the progress of mentees in achieving their goals.
- Provides feedback to both mentors and mentees based on progress, challenges, and outcomes.

Mentor Feedback and Evaluation

- Allows mentees to rate their mentors and provide feedback.
- Collects performance reviews to ensure mentors are effectively supporting mentees.

Mentorship Resources

- Provides resources, templates, and guides to support both mentors and mentees in their relationship.
- Includes best practices for mentoring, learning resources, and relevant career development tools.

Mentor Availability and Scheduling

- Allows mentors to specify their availability, preferred communication methods, and preferred mentee profiles.
- Ensures mentees can choose mentors whose availability and skills match their needs.

Mentorship Relationship Tracking

- Monitors the relationship and interaction between mentors and mentees, tracking the number of sessions, progress, and outcomes.

```
    // 7. Provide mentor evaluation and feedback

    boolean evaluateMentor(String menteeId, String mentorId, String evaluation);

    // 8. Offer mentorship resources and guidelines

    List<String> provideMentorshipResources(String mentorId, String menteeId);

    // 9. Track mentor availability and preferences

    boolean updateMentorAvailability(String mentorId, boolean isAvailable, String availableTime);

    // 10. Monitor and track the mentorship relationship

    MentorshipStatus trackMentorshipStatus(String mentorId, String menteeId);

}
```

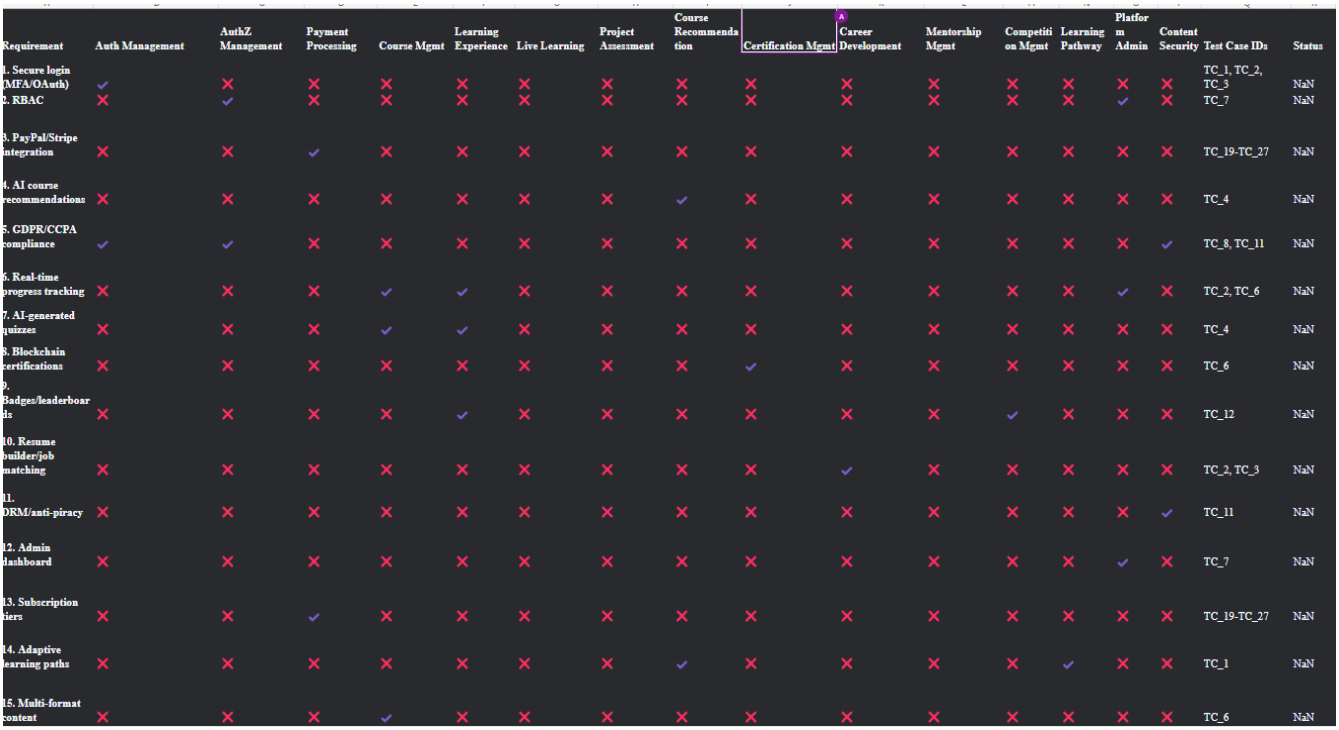
12. Competition Management	**Competition Creation**
	• Allows administrators or organizers to create and customize competitions, including setting rules, deadlines, and participation criteria.
	Participant Registration
	• Allows users to register for competitions by providing necessary details, such as their name, contact information, and prior experience (if required).
	Competition Challenges Management
	• Enables the creation, management, and scoring of individual challenges within a competition, including difficulty levels, scoring criteria, and problem statements.
	Leaderboard Management
	• Automatically generates and updates a leaderboard based on participants' performance during the competition.
	Real-Time Scoring and Results
	• Provides real-time scoring to track participants' progress and rank them accordingly throughout the competition.
	Prize Distribution
	• Facilitates the management of prizes and rewards for top performers, including physical prizes, monetary rewards, or certificates.
	Event Notifications and Reminders
	• Sends out notifications to participants about deadlines, important updates, and results.
	Competition History and Reports
	• Provides historical data on past competitions, such as winner profiles, challenges faced, and detailed reports.
		public interface CompetitionManagementSystem {
		boolean createCompetition(Competition competition);
		boolean registerForCompetition(String participantId, String competitionId);
		boolean createChallenge(String competitionId, Challenge challenge);
		boolean updateLeaderboard(String competitionId);
		boolean trackCompetitionProgress(String participantId, String competitionId);
		boolean assignPrizes(String competitionId);
		List<String> sendCompetitionNotifications(String competitionId);
		List<Competition> getCompetitionHistory();
		}

13. Platform Administration	1. **User Management**
	● Allows administrators to manage users, including adding, removing, and updating user profiles (e.g., learners, instructors, and administrators).
	2. **Role and Permission Management**
	● Enables the definition and management of user roles (admin, learner, instructor) and associated permissions.
	3. **Content Management**
	● Allows the uploading, updating, and management of course content, including videos, articles, quizzes, and assignments.
	4. **System Monitoring**
	● Provides real-time monitoring of system performance, including uptime, server status, and user activity.
	5. **Reporting and Analytics**
	● Generates reports on user activity, course performance, and overall platform metrics for insights and decision-making.
	6. **Platform Configuration**
	● Manages platform settings, including themes, branding, and other configurations that impact user experience.
	7. **Access Control**
	● Enforces security policies and ensures only authorized users have access to sensitive platform features.
	8. **Audit Logs**
	● Tracks and logs all administrative actions for accountability and compliance purposes.
		public interface PlatformAdminSystem {
		boolean addUser(User user);
		boolean removeUser(String userId);
		boolean assignRoleToUser(String userId, String role);
		boolean uploadContent(String contentId, Content content);
		boolean updateContent(String contentId, Content content);
		boolean generatePlatformReports();
		boolean configurePlatformSettings(PlatformSettings settings);
		List<AuditLog> getAuditLogs();
		}

14. Content Security System	**Access Control**
	• Manages and enforces access policies to protect sensitive content, ensuring only authorized users can view, edit, or download content.
	Content Encryption
	• Encrypts sensitive course materials and personal data to ensure privacy and security.
	Digital Rights Management (DRM)
	• Protects intellectual property by preventing unauthorized distribution or modification of content.
	Watermarking
	• Adds invisible or visible watermarks to content to deter unauthorized sharing and to track the source of leaks.
	User Authentication
	• Ensures only authenticated users can access restricted content by using login credentials or multi-factor authentication (MFA).
	Content Auditing
	• Tracks and monitors who accesses, downloads, or edits content, providing audit logs for security reviews.
	Data Backup and Recovery
	• Ensures secure backup of content and data, with the ability to recover lost content due to data corruption or system failure.
	Access Logs
	• Maintains logs of who accessed content, what actions they performed, and when, for security and auditing purposes.
		public interface ContentSecuritySystem {
		boolean encryptContent(String contentId);
		boolean applyDigitalRightsManagement(String contentId);
		boolean watermarkContent(String contentId);
		boolean authenticateUser(String userId, String password);
		boolean logAccess(String userId, String contentId, String action);
		boolean backupContent(String contentId);
		boolean restoreContent(String backupId);
		List<AccessLog> getAccessLogs();
		}

Traceability Matrix

LinkMtrix_link



Requirement	Auth Management	AuthZ Management	Payment Processing	Course Mgmt	Learning Experience	Live Learning	Project Assessment	Course Recommendation	Certification Mgmt	Career Development	Mentorship Mgmt	Competition Mgmt	Learning Pathway	Platform Admin	Content Security	Test Case IDs	Status
1. Secure login (MFA/OAuth)	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_1, TC_2, TC_3	NaN
2. RBAC	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	TC_7	NaN
3. PayPal/Stripe integration	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_19-TC_27	NaN
4. AI course recommendations	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	TC_4	NaN
5. GDPR/CCPA compliance	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	TC_8, TC_11	NaN
6. Real-time progress tracking	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	TC_2, TC_6	NaN
7. AI-generated quizzes	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_4	NaN
8. Blockchain certifications	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	TC_6	NaN
9. Badges/leaderboards	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	TC_12	NaN
10. Resume builder/job matching	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	TC_2, TC_3	NaN
11. DRM/anti-piracy	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	TC_11	NaN
12. Admin dashboard	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	TC_7	NaN
13. Subscription tiers	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_19-TC_27	NaN
14. Adaptive learning paths	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓	✗	✗	TC_1	NaN
15. Multi-format content	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_6	NaN

Test Requirements &Test Cases:

User Authentication Management

- The system shall allow users to register and log in using email, password, and social media accounts.
- The system shall support Multi-Factor Authentication (MFA) and OAuth-based login.
- The system shall manage user sessions securely, with automatic session expiration policies based on user roles.
- The system shall enforce data privacy regulations (GDPR, CCPA, etc.) and allow users to export or delete their data.
- The system shall implement suspicious activity detection & session revocation for security.

- The system shall implement Role-Based Access Control (RBAC) with custom permissions for students, instructors, and administrators.
- The system shall allow users to create and manage personal profiles, discussion posts, and activity feeds.
- The system shall maintain audit logs & security tracking for user actions and data access.

Test Cases for Authentication

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_1	User Registration	User registers with email and password	User is not registered	1. Open registration page 2. Enter email & password 3. Submit form	Valid email & password	User account is created successfully	User can log in		
TC_2	User Login	User logs in with valid credentials	User is registered	1. Open login page 2. Enter email & password 3. Click login	Correct credentials	User logs in successfully	User session starts		
TC_3	Multi-Factor Authentication (MFA)	User enables MFA	User is logged in	1. Open security settings 2. Enable MFA 3. Confirm MFA setup	Valid authentication setup	MFA is enabled successfully	MFA required for future logins		
TC_4	OAuth Login	User logs in using social media	User has a linked social account	1. Click "Login with Google" 2. Authenticate via Google 3. Grant permissions	Valid Google account	User logs in successfully	User session starts		
TC_5	Session Management	User session expires	User is logged in	1. Stay inactive for session timeout period 2. Attempt an action	Inactive session	User is logged out automatically	Redirected to login page		
TC_6	Suspicious Activity Detection	System detects login from unusual location	User logs in from an unknown location	1. Attempt login from new location 2. Enter credentials	New location login	System prompts for verification	User must verify identity		
TC_7	Role-Based Access Control (RBAC)	Admin accesses restricted settings	Admin is logged in	1. Navigate to admin dashboard 2. Attempt access to restricted settings	Admin credentials	Admin successfully accesses settings	Access granted		
TC_8	Data Privacy	User tries to delete another user's account	User is logged in	1. Navigate to another user's profile 2. Attempt deletion	Another user's account	System denies request	Error message displayed		
TC_9	Password Recovery	User enters unregistered email for reset	User has no registered email	1. Click "Forgot Password" 2. Enter unregistered email 3. Submit request	Invalid email	System rejects request	Error message displayed		

TC_10	Account Lockout	System does not lock out user after multiple failed attempts	User has an account	1. Enter incorrect password multiple times 2. Attempt login	Multiple incorrect attempts	System fails to lock account	User can still attempt login		

Customer Requirements

Career Development System

1. Users should be able to create and update their career profiles easily.
2. The system must support job applications and resume uploads.
3. Career counseling sessions should be bookable within the platform.
4. Users should have access to career development courses and enroll seamlessly.
5. Upon course completion, users should receive certifications automatically.
6. Skill assessments should be available to help users evaluate their competencies.
7. The system must provide personalized job recommendations based on user profiles.
8. Live webinars and training sessions should be accessible with real-time notifications.
9. The platform should be user-friendly with a responsive design.
10. Data security and privacy must be ensured for all user information.
11. **Career Support & Job Matching**
    - The system shall provide a Resume & CV Builder with AI-powered formatting suggestions.
    - The system shall implement AI-based job & internship matching based on student skills & completed courses.
    - The system shall allow employers to create job listings & search for talent.
    - The system shall offer company-sponsored courses linked to job opportunities.

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_1	User Profile Management	User creates a	User has access to the system	1. Open profile creation page	Valid user details	Profile is successfully created and saved	Profile appears in user dashboard		

		career profile		2. Enter required details 3. Submit profile					
TC_2	Job Application Process	User applies for a job	User has an active profile	1. Browse available jobs 2. Select a job 3. Submit application	Job details, resume	Application is successfully submitted	Application appears in job history		
TC_3	Resume Management	User uploads resume	User is logged in	1. Navigate to resume section 2. Upload resume file 3. Save changes	Valid resume file	Resume is successfully uploaded	Resume is stored in user profile		
TC_4	Career Counseling	User books a counseling session	User is logged in	1. Open counseling section 2. Choose an available slot 3. Confirm booking	Session details	Session is successfully booked	Booking confirmation is displayed		
TC_5	Course Enrollment	User enrolls in a career course	User is logged in	1. Browse courses 2. Select a course 3. Enroll	Course details	User is successfully enrolled	Course appears in enrolled courses list		
TC_6	Certification Issuance	User completes a course and requests certification	User has completed all course modules	1. Verify course completion 2. Request certification 3. Download certificate	Completed course details	Certificate is issued successfully	Certificate available in user dashboard		
TC_7	AI Career Advice	User requests AI-driven career suggestions	User has an incomplete profile	1. Open AI career section 2. Input career preferences	Missing profile data	AI cannot generate recommendations	Error message displayed		
TC_8	Employer Job Posting	Employer posts a job	Employer has an active company account	1. Open job posting section 2. Enter job details 3. Submit job post	Valid job details	Job is successfully posted	Job appears in listings		
TC_9	Live Meeting	User joins a scheduled webinar	Webinar is available	1. Navigate to webinars 2. Click join 3. Enter session	Webinar details	User successfully joins the webinar	Webinar session starts		
TC_10	Skill Assessment	User takes a career skill test	User loses internet connection	1. Start assessment 2. Answer questions 3. Submit test	Assessment questions	Submission fails due to connectivity issues	Test not recorded		

Customer Requirements

Course Management

1. The system shall allow instructors to create, edit, and organize courses with structured modules.
2. The system shall track student progress in real-time and provide adaptive difficulty settings for assessments.
3. The system shall support AI-assisted course creation (e.g., auto-generating quizzes from content).
4. The system shall provide multi-format content delivery (videos, PDFs, SCORM packages, interactive media).
5. The system shall issue certifications upon course completion, with public verification links.
6. The system shall support course versioning, allowing instructors to update content without affecting enrolled students.
7. The system shall allow students to submit assignments and receive instructor feedback.
8. The system shall provide discussion forums and interactive Q&A for collaborative learning.
9. The system shall integrate plagiarism detection for submitted assignments.
10. The system shall ensure role-based access control (RBAC) for students, instructors, and administrators.
11. The system shall support course enrollment limits to prevent overbooking.
12. The system shall allow instructors to schedule live sessions and notify students.
13. The system shall enable students to bookmark important content for quick access.
14. The system shall provide analytics dashboards for instructors to track student performance.

Test Cases for Course Management System

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_1	Course Creation	Instructor creates a new course	Instructor has access to course management	1. Open course creation module 2. Add modules 3. Save course	Valid course details	Course is successfully created and visible to students	Course is listed in course catalog		
TC_2	Student Progress Tracking	Student checks progress	Student is enrolled in a course	1. Complete lessons 2. Check progress tracker	Completed lessons count	Progress is updated in real-time	Updated progress shown		

TC_3	Adaptive Assessments	Student takes adaptive quiz	Student has access to quizzes	1. Start adaptive quiz 2. Answer questions	Varying difficulty questions	Quiz difficulty adjusts based on performance	Next quiz adapts to responses		
TC_4	AI Quiz Generation	Instructor enables AI quiz	AI quiz generation is enabled	1. Enable AI quiz generation 2. AI generates questions	Course content	AI-generated questions are relevant	AI-generated quiz created		
TC_5	Certification Issuance	Student completes a course	Course completion criteria met	1. Complete all course modules 2. Request certification	Completed course	Certificate is issued with verification link	Certificate generated		
TC_6	Assignment Submission	Student submits an assignment	Student is enrolled in a course	1. Open assignment page 2. Upload assignment 3. Submit	Valid assignment file	Assignment is successfully submitted	Assignment recorded		
TC_7	Live Session Scheduling	Instructor schedules live session	Instructor has access to course tools	1. Open live session module 2. Set schedule 3. Notify students	Conflicting schedule details	System rejects due to schedule conflict	No session created		

Payments & Monetization

- The system shall integrate with multiple **payment gateways (PayPal, Stripe, etc.)** for seamless transactions.
- The system shall support **subscription-based plans, one-time purchases, and tiered access**.
- The system shall enable administrators to create and manage **discounts, promo codes, and referral rewards**.

Test Requirements:

1. Payment Gateway Integration

1.1 The system shall integrate with multiple payment gateways, including PayPal and Stripe.

1.2 The system shall validate payment details and reject invalid or expired payment information.

1.3 The system shall log all payment transactions for auditing purposes.

2. Subscription, One-Time, and Tiered Purchases

2.1 The system shall support subscription-based plans with automatic renewals.

2.2 The system shall allow users to make one-time purchases.

2.3 The system shall provide tiered access levels with different feature sets.

3. Discounts, Promo Codes, and Referral Rewards

3.1 The system shall allow administrators to create, manage, and disable promo codes.

3.2 The system shall apply valid promo codes and display appropriate messages for invalid ones.

3.3 The system shall prevent abuse of promo codes and referral rewards.

Test cases :

1. Payment Gateway Integration

Test case num	*senario*	*Test data*	*steps*	*Expected out put*
TC-01	***1.1***	Verify that PayPal is available as a payment option	1. Navigate to the checkout page 2. Select payment method options 3. Check for PayPal option	PayPal is displayed as an available payment method

TC-02	*1.1*	Verify that Stripe is available as a payment option	1. Navigate to the checkout page 2. Select payment method options 3. Check for Stripe option	Stripe is displayed as an available payment method
TC-03	*1.1*	Verify that the system correctly redirects users to the selected payment gateway.	1. Select PayPal or Stripe as payment method 2. Click on "Proceed to Payment" 3. Observe redirection	User is redirected to the selected payment gateway
TC-04	*1.2*	Verify that the system rejects payments with invalid card numbers.	1. Select credit card payment 2. Enter an invalid card number 3. Attempt to process payment	Payment is declined, and an error message is displayed
TC-05	*1.2*	Verify that the system rejects payments with expired card details	1. Select credit card payment 2. Enter an expired card 3. Attempt to process payment	Payment is declined, and an error message is displayed
TC-06	*1.2*	Verify that the system rejects payments with incorrect billing addresses.	1. Select credit card payment 2. Enter valid card details but incorrect billing address 3. Attempt to process payment	Payment is declined, and an error message is displayed

TC-07	*1.3*	Verify that users receive real-time confirmation for successful transactions	1. Complete a successful payment 2. Observe confirmation message	Real-time confirmation is displayed
TC-08	*1.3*	Verify that users receive real-time failure notifications for declined transactions.	1. Attempt a payment with invalid details 2. Observe system response	Real-time failure message is displayed
TC-09	*1.3*	Verify that transaction confirmation details include transaction ID, amount, and payment method	1. Complete a successful transaction 2. Check the confirmation details	Email notification is received with transaction details

2. Subscription, One-Time, and Tiered Purchases

Test case num	*senario*	*Test data*	*steps*	*Expected out put*
TC-10	*2.1*	Verify that users can subscribe to a plan with automatic renewal	1. Navigate to the subscription page 2. Select a monthly subscription plan 3. Enter valid payment details 4. Confirm the subscription	Subscription is activated with automatic renewal
TC-11	*2.1*	Verify that users are charged automatically on the renewal date	1. Subscribe to a plan with auto-renewal 2. Wait until the next billing cycle	Payment is processed, and subscription is renewed

			3. Check if the payment is processed automatically	
TC-12	2.1	Verify that users can cancel their subscription before renewal.	1. Subscribe to a monthly plan 2. Navigate to subscription settings 3. Click on "Cancel Subscription" 4. Confirm cancellation	Subscription is canceled, and auto-renewal is disabled
TC-13	2.2	Verify that users can make one-time purchases	1. Navigate to the store 2. Select a one-time purchase item 3. Enter valid payment details 4. Confirm purchase	One-time purchase is successful, and item is available
TC-14	2.2	Verify that one-time purchases do not renew automatically.	1. Purchase an item as a one-time purchase 2. Wait for a billing cycle 3. Check if a charge occurs	No additional charges are made after the purchase
TC-15	2.2	Verify that users can access one-time purchases after payment.	1. Purchase an item 2. Navigate to the user's library 3. Check if the item is available	Purchased item is accessible to the user
TC-16	2.3	verify that users can choose different tiered plans.	1. Navigate to the subscription page	Different tiered plans are available with distinct feature sets

			2. View the available plans 3. Compare features 4. Select a plan	
TC-17	*2.3*	Verify that upgrading from a lower-tier plan to a higher-tier plan is possible.	1. Subscribe to the Basic plan 2. Navigate to the upgrade section 3. Select the Premium plan 4. Confirm upgrade	Plan is upgraded successfully, and new features are unlocked
TC-18	*2.3*	Verify that users are charged the correct amount based on the selected tier.	1. Subscribe to different plans 2. Check the payment summary 3. Confirm transaction amount	User is charged based on the selected plan price

3. Discounts, Promo Codes, and Referral Rewards

Test case num	*senario*	*Test data*	*steps*	*Expected out put*
TC-19	*3.1*	Verify that administrators can create new promo codes.	1. Log in as admin. 2. Navigate to promo code management. 3. Create a new promo code. 4. Save changes	Promo code is successfully created.
TC-20	*3.1*	Verify that administrators can update existing promo codes	1. Log in as admin. 2. Edit an existing promo code. 3. Modify discount.	Promo code is updated successfully.

			4. Save changes.	
TC-21	3.1	Verify that administrators can disable promo codes	1. Log in as admin. 2. Disable a promo code. 3. Save changes.	Promo code is disabled successfully.
TC-22	3.2	Verify that users can enter and apply a valid promo code.	1. Add items to cart. 2. Enter a valid promo code. 3. Apply the code	Discount is applied successfully.
TC-23	3.2	Verify that the system correctly applies the discount for a valid promo code.	1. Apply a valid promo code. 2. Check the total amount.	Discounted price is displayed correctly
TC-24	3.2	Verify that users receive an appropriate error message when entering an invalid promo code.	1. Attempt to apply an invalid promo code.	System displays an error: "Invalid promo code."
TC-25	3.3	Verify that a user cannot use the same promo code multiple times if not allowed.	1. Apply a promo code once. 2. Attempt to apply the same code again	System prevents re-use and displays an error.
TC-26	3.3	Verify that users cannot use multiple promo codes on the same purchase if restricted	1. Apply the first promo code. 2. Attempt to apply another code.	System prevents stacking of promo codes.

TC-27	3.3	Verify that referral rewards are only granted for unique and valid referrals	1. User A refers User B. 2. User B signs up and makes a purchase.	User A receives referral reward.

Test Cases for Manage RoadMap:

Test Case Num	Scenario	Test Steps	Test Data	Expected Output
TC-01	Verify RoadMap Accessibility	1. Log in as a student; 2. Navigate to the dashboard; 3. Click on "View RoadMap"	Valid student credentials	RoadMap is displayed with clear sections for milestones and modules.
TC-02	Verify RoadMap Content Accuracy	1. Open the RoadMap; 2. Review listed courses and milestones	Current course data and progress	RoadMap accurately shows enrolled courses, prerequisites, and milestones.
TC-03	Verify Dynamic Update After Module Completion	1. Complete a course module; 2. Refresh the RoadMap	Completed module details	RoadMap updates to mark the module as completed and adjusts remaining milestones.
TC-04	Verify RoadMap Recommendation Functionality	1. Complete a module; 2. Check recommendations on the RoadMap	Module completion data	System suggests next courses based on learning history and career goals.

Test Cases for Manage Career Support:

Test Case Num	Scenario	Test Steps	Test Data	Expected Output
TC-06	Verify Career Support Accessibility	1. Log in as a student 2. Navigate to the "Career Support" section	Valid student credentials	Career Support dashboard is accessible and displays available tools
TC-07	Verify Resume Builder Functionality	1. Open the resume builder 2. Input required details 3. Submit to generate resume	Personal and educational details	A formatted, downloadable resume is generated
TC-08	Verify Personalized Job Matching	1. Access job recommendations 2. Review suggested jobs based on profile	Updated student profile and skills	Job recommendations list is tailored to the student's profile and course history
TC-09	Verify Career Counseling Booking	1. Navigate to counseling section 2. Select an available time slot 3. Confirm booking	Counseling session slots	Counseling session is successfully booked and confirmation is displayed

UI Link: link